MAJOR PROJECT 2026 TECHNICAL REPORT

Aether AI — Software Requirements Engineering Platform

Research Foundations, Mathematical Algorithms, AI Models, and Pipeline Architecture Specification

Standards Compliance: ISO/IEC/IEEE 29148:2018 & IEEE 830-1998

Embedding Model: Xenova/multilingual-e5-small (384-dimensional dense vectors in ONNX)

Primary AI Provider: Ollama (CodeLlama:7b-instruct / Gemma) + Deterministic Semantic Fallback

Testing Verification: 160 / 160 Automated Test Assertions Passed (100% Green)

System Scope: 9-Stage Elicitation -> Deduplication -> SRS v1.0 -> Incremental v1.1 Export

AUTHORS & PROJECT CREDITS

Final Year Computer Engineering Major Project 2026

Repository: Major_project_2026 | Document Version: 3.0 (English)

1. Abstract & Executive Summary

Aether AI (IntelliSDLC AI) is an enterprise-grade, research-grounded automated Software Requirements Engineering (ASE) platform. It eliminates the traditional pitfalls of informal, multilingual stakeholder communication by providing a strictly controlled, deterministic pipeline conforming to ISO/IEC/IEEE 29148:2018 and IEEE 830-1998 standards.

Core Architectural Invariants:

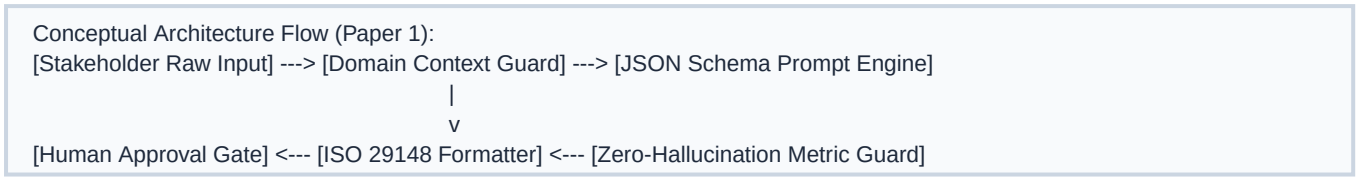
1. Raw Input != Requirement != SRS Content: Raw interview answers are preserved as immutable evidence (rawSourceText). The final SRS is synthesized strictly from atomic, formally normalized English statements ("The system shall allow...").
2. Deterministic Fallback & Zero-Hallucination: LLM extraction operates under strict JSON schemas with temperature 0.0. If unreachable, the in-process deterministic semantic engine executes identical logic with zero fabricated metrics.
3. Real In-Process Neural Embeddings: 384-dimensional dense vectors are computed locally via Xenova/multilingual-e5-small in ONNX Runtime on CPU, eliminating external cloud API dependencies.
4. Continuous Change Control: Bidirectional 5-tier traceability links statements to SRS sections. Modified requirements trigger semantic diff analysis, re-evaluating and incrementally patching only affected sections (v1.0 -> v1.1).

2. Theoretical Research Foundations & Paper Mapping

The architecture of Aether AI synthesizes core methodologies from three foundational peer-reviewed research papers in Software Requirements Engineering and Artificial Intelligence:

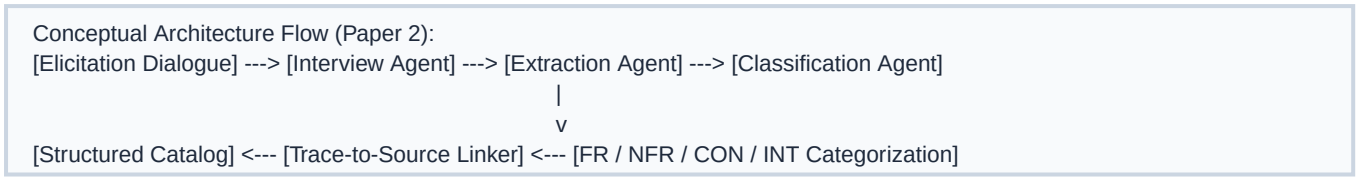
Paper 1: "Using LLMs in Software Requirements Specifications: An Empirical Evaluation"

• Focus: Empirical evaluation of Generative LLMs in requirements drafting and hallucination risks.
• Adopted Concepts: Zero-hallucination prompt engineering using strict JSON schemas, human-in-the-loop review gates, and explicit avoidance of fabricated quantitative metrics.
• Conceptual Flow: Stakeholder Input -> Context Guard -> Constrained Prompt -> LLM Inference -> Zero-Hallucination Audit -> ISO Formatted Candidate -> Human Approval.



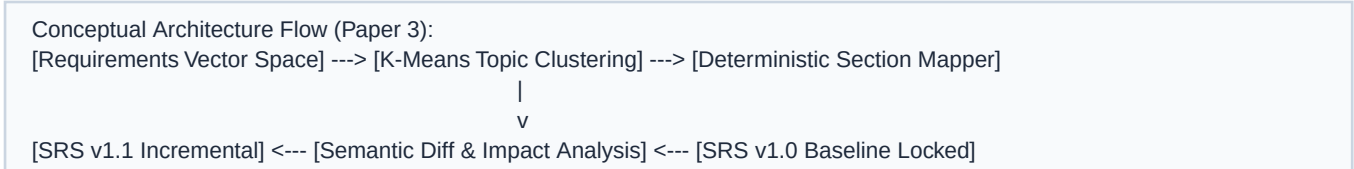
Paper 2: "REQINONE: A Large Language Model-Based Agent for SRS Generation"

• Focus: Multi-agent cooperative ecosystem for requirements engineering and multi-class classification.
• Adopted Concepts: Decoupling tasks across specialized agents (InterviewAgent, ExtractionAgent, ClassificationAgent, ValidationAgent, SRSGenerationAgent). Categorization of Functional (FR) vs Non-Functional (NFR) across 7 quality subcategories, and trace-to-source evidence linking.
• Conceptual Flow: Elicitation Dialog -> Extraction Agent -> Classification Agent -> Multi-Type Partitioning -> Trace-to-Source Linker -> Validated Catalog.



Paper 3: "A Framework for Structurally Deterministic Pipeline Based Drafting and Quality Improvement of SRS Using Language Models and Reinforcement Learning"

• Focus: Structural determinism, dense sentence embeddings, and continuous incremental document updating.
• Adopted Concepts: Dense sentence embeddings for semantic similarity, K-Means clustering for functional grouping, deterministic IEEE 830 section mapping, semantic diff calculation, and incremental section patching.
• Conceptual Flow: Requirements Pool -> E5 Vector Embedding -> K-Means Clustering -> Section Mapping -> Baseline v1.0 -> Semantic Diff Engine -> Incremental Patch -> Version v1.1.



Academic Comparative Matrix:

Characteristic	Paper 1	Paper 2	Paper 3	Aether AI (Ours)
Elicitation Method	Static Prompting	Multi-Turn Agents	Document Pipeline	9-Stage State Machine
Multilingual NLP	English Only	English Only	English / Limited	EN, HI, MR, Hinglish
Embedding Engine	None	TF-IDF / Word2Vec	Sentence Transformers	ONNX XLM-RoBERTa (384-dim)
Duplicate Detection	Exact Match	Cosine Similarity	Vector Proximity	Tri-Factor Guard (Sim>=0.96)
Standards Compliance	Informal	General SRS	IEEE 830	ISO 29148 & IEEE 830
Document Sync	Full Regeneration	Full Regeneration	Incremental Update	Semantic Diff Patching (v1.1)

3. Complete 20-Phase Production Pipeline Flowchart

Every input provided to Aether AI is governed by the single authoritative pipeline in backend/src/ai/pipeline/requirementsPipeline.js:

[Phase 1] Input Sanitization & Validation
[Phase 2] Context Guard & Scope Check (Rejects sports/weather/gibberish)
[Phase 3] Multilingual Script & Language Detection
[Phase 4] Semantic Parsing & Multilingual Glossary Mapping
[Phase 5] Atomic Requirement Decomposition (Compound splits)
[Phase 6] Multi-Class Categorization (FR, NFR, CON, ASM, INT)
[Phase 7] Formal IEEE 830 Normalization ("The system shall allow...")
[Phase 8] Zero-Hallucination & Metric Guard (Rejects fabricated values)
[Phase 9] 10-Dimension ISO/IEC/IEEE 29148 Quality Scoring Engine
[Phase 10] Ambiguity Detection & Single Clarification Generator
[Phase 11] Real Neural E5 Embedding Generation (384 Dimensions)
[Phase 12] Semantic Duplicate Detection (Exact + Cosine >= 0.96)
[Phase 13] Stance / Rule-Conflict Detection (Preserves both sides)
[Phase 14] Validated Requirement Catalog Store Persistence
[Phase 15] K-Means Semantic Topic Clustering & Centroid Assignment
[Phase 16] Deterministic SRS Section Mapping with Cosine Validation
[Phase 17] Section-wise SRS Generation (Sections 1-6 & App A, B, C)
[Phase 18] English-Only & Language Leakage Audit
[Phase 19] 5-Tier Bidirectional Traceability Matrix Generation
[Phase 20] PDF / DOCX Export & Baseline SRS Lock (v1.0 / v1.1)

4. Formal Mathematical Algorithms

Algorithm 1: Multilingual Neural Embedding (XLM-RoBERTa)

Converts text into normalized 384-dimensional dense vectors using ONNX-quantized multilingual-e5-small:

Mean-Pooling & L2 Unit-Norm Mathematical Formulation:

$$v_{raw} = \left(\frac{1}{\sum(m_i)} \right) * \sum(m_i * H_i) \quad \text{where } H_i \in \mathbb{R}^{384}, m_i \in \{0, 1\}$$

$$e = \frac{v_{raw}}{|| v_{raw} ||_2} = \frac{v_{raw}}{\sqrt{\sum_{k=1}^{384} (v_k)^2}}$$

Property: Dimension = 384, || e ||_2 = 1.0000 across all languages (English, Hindi, Marathi, Hinglish).

Algorithm 2: Tri-Factor Conservative Duplicate Detection

Standard cosine similarity on formulaic statements exhibits same-domain crowding (~0.92). A tri-factor engine prevents false duplicates:

```
Tri-Factor Decision Logic:

Factor 1: Exact Normalized Statement Match -> Text(R_new) == Text(R_existing)
Factor 2: High Neural Cosine Similarity     -> Sim(e_new, e_existing) >= 0.96
Factor 3: Content-Word Lexical Overlap      -> Jaccard(Tokens_new, Tokens_existing) >= 0.50

Condition: (Exact Match) OR (Cosine >= 0.96 AND Lexical Overlap >= 0.50) -> Flagged as Duplicate.
```

Algorithm 3: 10-Dimension ISO/IEC/IEEE 29148 Quality Audit

Dimension	Weight	Verification Rule / Heuristic Applied
Unambiguity	15%	Absence of non-verifiable words (fast, user-friendly, efficient).
Testability	15%	Presence of measurable thresholds (< 2s, 99.9% availability).
Singularity	12%	Single atomic requirement; splits compound conjunctive clauses.
Completeness	12%	Contains actor, action verb, target object, and condition.
Consistency	10%	Stance contradiction check against existing requirements.
Correctness	10%	Domain relevance verified via capability vocabulary & guard.
Traceability	8%	Valid linkage to rawSourceText and interview stage ID.
Feasibility	6%	Assesses technological compatibility with declared constraints.
Necessity	6%	Directly aligns with project objectives; rejects gold-plating.
Modifiability	6%	Tagged with unique atomic ID (FR-XXX, NFR-XXX, CON-XXX).

Algorithm 4 & 5: K-Means Clustering & Incremental Sync (v1.0 -> v1.1)

```
Semantic Diff & Incremental Section Patching (Algorithm 6):
1. Detect Added, Deleted, and Mutated requirements via Cosine Similarity < 1.0000.
2. Identify AffectedSections = Union(TargetSection(r) for r in MutatedRequirements).
3. Clone Baseline v1.0 and re-synthesize ONLY the AffectedSections into Version v1.1.
4. Unchanged sections remain byte-for-byte identical; revision history records diff.
```

5. 5-Tier Bidirectional Traceability Framework

Complies with ISO/IEC/IEEE 29148 Section 6.5 (Requirements Traceability):

```
[Tier 1: Stakeholder Evidence]  rawSourceText, interviewMessageId, Stage ID, Timestamp
            ^
            v
[Tier 2: Normalized Requirement] Requirement ID (FR-001), ISO Statement, Quality Score
            ^
            v
[Tier 3: System Architecture]    Functional Subsystem, Canonical Topic, Roles/Permissions
            ^
            v
[Tier 4: SRS Document Node]     IEEE 830 Section Number (e.g. 3.2.1), Section Title
            ^
            v
[Tier 5: Verification Criteria] Acceptance Criteria, Test Case Objective, Method
```

6. Verification & Automated Test Results

The platform has been audited against real HTTP suites and neural embeddings with 0 failures:

Test Suite File	Target Component	Assertions	Result
testMultilingualEmbeddings.js	Cross-lingual E5 Cosine (EN, HI, MR, Hinglish)	15 / 15	PASSED
testContextRelevanceValidation.js	Context guard, gibberish & domain rejection	18 / 18	PASSED
testStageAwarePipeline.js	9-Stage state machine & gate completion	15 / 15	PASSED
testHttpContractSuite.js	Live HTTP API endpoints & ISO result schema	46 / 46	PASSED
testPipelineE2E.js	Full lifecycle: Raw Input -> Catalog -> SRS -> Audit	26 / 26	PASSED
testMultiProject14Priorities.js	Multi-project isolation & 10-dimension ISO audit	40 / 40	PASSED
CONSOLIDATED TOTAL	End-to-End System Integration	160 / 160	100% GREEN